for its predictions.

To verify this, we conduct the same intervention experiment as Li et al. (2023a). Namely, given an input game sequence (and its corresponding board state $B$), we intervene to make the model believe in an altered board state $B'$. We then observe whether the model's prediction reflects the made-believe board state $B'$ or the original board state $B$.

Our intervention approach is simple: we add our linear vectors to the residual stream of each layer:

$$x' \leftarrow x + \alpha p_d^{\lambda}(x)$$

where $d$ indicates a direction amongst {MINE, YOURS, EMPTY} and $\alpha$ is a scaling factor. In other words, to flip a tile from YOURS to MINE, we simply push the residual stream at every layer in the MINE direction, or to "erase" a previously played tile, we push in the EMPTY direction. [4] [5]

Note that this intervention is much simpler than that of Li et al. (2023a). Namely, Li et al. (2023a) edits the activation space ($x$) of OthelloGPT using several iterations of gradient descent from their non-linear probe. Instead, we perform a single vector addition.

### 4.2 Experiment Setup

For our intervention experiment, we adopt the same setup and metrics as Li et al. (2023a). We use an evaluation benchmark consisting of 1,000 test cases. Each test case consists of a partial game sequence ($B$) and a targeted board state $B'$.

We measure the efficacy of our intervention by treating the task as a multi-label classification problem. Namely, we compare the top-$N$ predictions post-intervention against the groundtruth set of legal moves at state $B'$, where $N$ is the number of legal moves at $B'$. We then compute error rate, or the number of false positives and false negatives.

Li et al. (2023a) only considers the scenario of flipping the colour of a tile. To also validate our EMPTY direction, we also experiment with "erasing" a previously played tile by making it empty.

### 4.3 Results

Table 2 shows the average error rates after our interventions. Our interventions are equally effective as that of gradient-based editing, and confirms that our interpretation of each linear direction matches how the model uses such directions.

| Flipping colours | Avg. # Errors |
|---|---|
| Null Intervention Baseline | 2.723 |
| Non-Linear Intervention | 0.12 |
| Linear Probe Addition | **0.10** |
| **Erasing** | **Avg. # Errors** |
| Null Intervention | 2.73 |
| Non-Linear Intervention | 0.11 |
| Linear Probe Addition | **0.02** |

Table 2: Error rates from interventions.

## 5 Additional Linear Interpretations

The linear representation hypothesis is of interest to the mechanistic interpretability community because it provides a foothold into understanding a system. The internal state of the transformer, the residual stream, is the sum of the outputs of all previous components (heads, layers, embeddings and neurons) (Elhage et al., 2021), so any linear function of the residual stream can be linearly decomposed into contributions from each component, allowing us to trace back where a computation is coming from.

In this section we leverage our newfound linear representation of board state to provide additional interpretations of OthelloGPT, as proof of concept of how discovering linear representations unlocks downstream interpretability applications.

### 5.1 Interpreting Empty Tiles

Here we interpret how OthelloGPT derives the status of empty tiles.

**The EMPTY Circuit.** A key insight for EMPTY is that input tokens each correspond to a tile on the board (i.e., A4), and once played, the tile can only change colour but remains non-empty.

We view OthelloGPT as using attention heads to "broadcast" which moves have been played: given a move at timestep $t$, attention heads write this information into other residual streams. This information (PLAYED) can be represented as following. First, each move $m$ (A4) is embedded: $Emb[m]$. Then the model writes this information to other residual streams using linear projections $Att.V$ and $Att.O$ (Section 2.3):

[4] We experiment with intervening on different layers. See Appendix for more details.

[5] We use the TransformerLens library: https://github.com/neelnanda-io/TransformerLens.

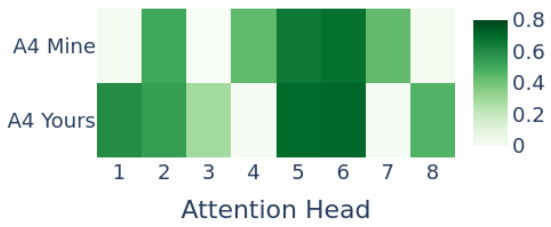


Figure 3: Difference in probability of A4 being empty, between our clean and corrupt sequences, measured in each attention head.

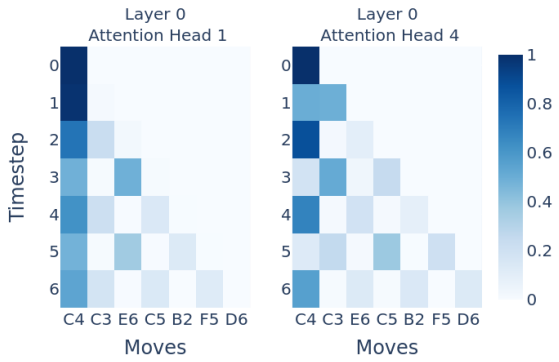


Figure 4: Examples of attention heads attending to YOUR (left) or MY (right) moves.

$$\text{PLAYED}_h(m) = Emb[m]@Att_h.V@Att_h.O$$

For each attention head in the first layer,[6] we compute the cosine similarity between PLAYED and the $p^{\lambda}_{\text{EMPTY}}$ direction:

$$\max_{h \in H} \text{CosSim}(\text{PLAYED}_h(m), p^{\lambda}_{\text{EMPTY}}(m))$$

Since the two terms encode *opposite* information, we expect a high negative cosine similarity.

We observe an average similarity score of **-0.862** across all 60 squares,[7], confirming that $p_{\text{EMPTY}}$ is encoding NOT PLAYED. This tells us that $p_{\text{EMPTY}}$ is a linear function of the token embeddings.

This also implies that OthelloGPT knows which tiles are empty by $x^{0_mid}$: after the first attention heads but before the MLP layer. On a binary classification task of EMPTY vs. NOT-EMPTY from 1,000 games in our test split, our probe achieves an accuracy of **76.8%** and **98.9%**, when projecting from $X^{0_pre}$ and $x^{0_mid}$ respectively.

**Logit Attribute for EMPTY.** The previous analysis is based on the *weights* of the model. Here we provide an alternative analysis by studying the *activations* during inference.

First, we select a move $m$ (A4) that we wish to explain. We then construct a "clean" and "corrupt" set of partial game sequences (N=4,569). Our clean set always includes $m$, while our corrupt set replaces all timesteps with $m$ in the clean set with an alternative move. We ensure that all games in our corrupt set remain legal sequences. Finally, we study the *difference in probability* that $m$ is empty, according to our probes, in our two sets. Namely, we project the outputs from each attention head onto the EMPTY direction and apply a softmax:

$$P_{\text{EMPTY}[m]}(\sigma) = Softmax(\sigma * p^{\lambda}_{\text{EMPTY}[m]})$$

where $\sigma$ is the output from each attention head.

Figure 3 shows the difference in probability that A4 is empty, between our clean and corrupt inputs, measured in each attention head of the first layer. The figure decomposes two scenarios: when A4 was originally played by ME or YOU. This is because some attention heads only attend to MY moves (4, 7), while some only attend to YOURS (1, 3, 8), which we show below.

### 5.2 Attending to MY & YOUR Timesteps

We find that some attention heads only attend to either MY or YOUR moves. Figure 4 shows two examples: at each timestep, each head *alternates* between attending to even or odd timesteps. Such behavior further indicates how the model computes its world model based on MINE and YOURS as opposed to BLACK and WHITE.

### 5.3 Additional Linear Concepts: FLIPPED

In addition to linearly representing the board state, we find that OthelloGPT also encodes which tiles are being flipped, or captured, at each timestep. To test this, we modify our probing task to classify between FLIPPED vs. NOT-FLIPPED, with the same training setup described above. Given the class imbalance, for this experiment we report $F1$ scores. Table 3 demonstrates high $F1$ scores by layer 3.

[6] Knowing which moves were PLAYED (i.e. show up in the input sequence), should not depend on any other computation, and thus we expect this information to be written by the attention heads in the first layer.

[7] The center 4 squares can never be empty.

| | $x^0$ | $x^1$ | $x^2$ | $x^3$ | $x^4$ | $x^5$ | $x^6$ | $x^7$ |
|---|---|---|---|---|---|---|---|---|
| Linear {FLIPPED, NOT-FLIPPED} | 74.76 | 85.75 | 91.62 | 94.82 | 96.44 | 97.13 | 96.82 | 96.3 |

Table 3: $F1$ score for probing on FLIPPED tiles. In addition to the board state, the model also linearly encodes concepts such as flipped tiles per timestep.

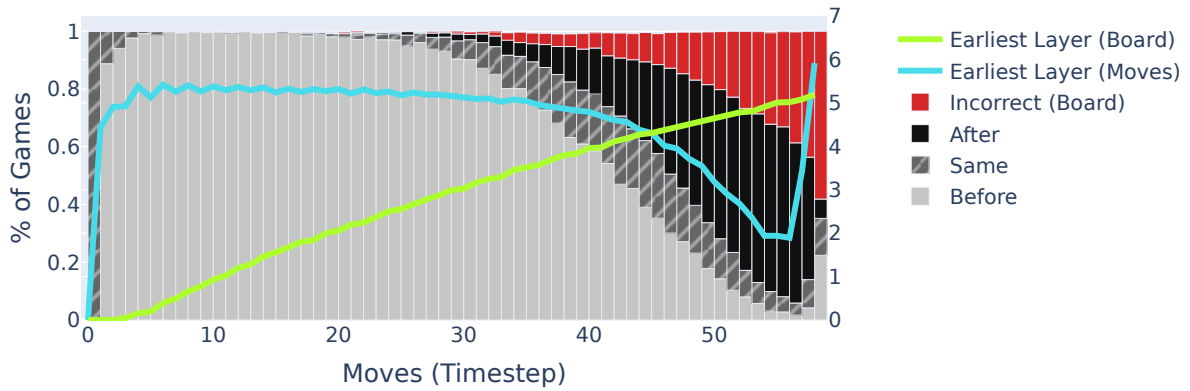


Figure 5: Proportion of times the board state is computed before/after move predictions are made (First y-axis). **Light Grey:** Boards are computed in an earlier layer than moves. **Dark Grey, Black:** Boards are computed in the same or later layer than moves. **Red:** Model never computes the correct board state. **Aqua, Lime (Curves):** Average earliest layer in which the board or moves are correctly computed (Second y-axis). Starting from the mid-game, we start observing the model compute moves before boards (black bar), and this occurs more frequently as the game progresses.

We also conduct a modified version of our intervention experiment, in which we always randomly select a flipped tile at the current timestep to intervene on. Then, instead of adding either $p^{\lambda}_{\text{MINE}}$, $p^{\lambda}_{\text{YOURS}}$, or $p^{\lambda}_{\text{EMPTY}}$, we *subtract* $p^{\lambda}_{\text{FLIPPED}}$. This tests whether the FLIPPED feature is causally relevant for computing the next move, by exploring whether this is sufficient to cause the model to play valid moves in the new board state. We get an average error rate of **0.486**, compared to a null intervention baseline rate of **1.686**.

One can consider FLIPPED tiles as the difference between the previous and current board state. One might naturally think that a recurrent computation could derive the current board state by iteratively applying such differences. However, transformer models do **not** make recursive computations![8] We view FLIPPED to be both an unexpected encoding and a hint for the rest of the board circuit.

### 5.4 Multiple Circuits Hypothesis

Although we find a board state circuit and its causality on move predictions, we find that it does not explain the entire model. If our understanding is correct, we expect the model to compute the board state before computing valid moves. However, we find that in end games, this is not the case.

To check for the correct board state, we apply our linear probes on each layer, and check the earliest layer in which all 64 tiles are correctly predicted.[9] To check for correct move predictions, we project from each layer using the unembedding layer, and check the earliest layer in which the top-N move predictions are all correct, where N is the number of groundtruth legal moves.

Figure 5 plots the proportion of times the board state is computed before (or after) valid moves (first y-axis). We also overlay the average earliest layer in which board or moves are correctly computed (second y-axis, aqua and lime curves). To our surprise, we find that in end games, the model often computes legal moves *before* the board state (black bars). We henceforth refer to this behavior as MOVEFIRST, and share some thoughts.

**End Game Circuits.** First, MOVEFIRST starts to occur around move 30, which is the mid-point of the game. Second, MOVEFIRST occurs more frequently as we near the end of the game (increasing black bars). Interestingly, in Othello, starting from

[8] Doing so would require our transformer model to have the same number of layers as the maximum game sequence length of 60.

[9] It might be the case that legal moves could be predicted without 100% accuracy of the board state. We try variants (see Appendix), but observe similar trends.